

UNIT III – MEMORY SYSTEM ARCHITECTURE

Memory Hierarchy Overview: Registers to Secondary Storage, RAM Types: SRAM, DRAM, Flash –Industry Applications, Cache Memory: Structure and Mapping Techniques, Cache



Write Policies and Replacement Algorithms, Multilevel Caches and Performance Tuning, Virtual Memory: Paging, TLB, and Address Translation , Memory Management in OS (Real world Focus: Linux)

MEMORY HIERARCHY OVERVIEW

Memory Hierarchy helps in optimizing the memory available in the computer. There are multiple levels present in the memory, each one having a different size, different cost, etc. Some types of memory like cache, and main memory are faster as compared to other types of memory but they are having a little less size and are also costly whereas some memory has a little higher storage value, but they are a little slower. Accessing of data is not similar in all types of memory, some have faster access whereas some have slower access.

Types of Memory Hierarchy

This Memory Hierarchy Design is divided into 2 main types:

- **External Memory or Secondary Memory:** Comprising of Magnetic Disk, Optical Disk, and Magnetic Tape i.e. peripheral storage devices which are accessible by the processor via an I/O Module.
- **Internal Memory or Primary Memory:** Comprising of Main Memory, Cache Memory & CPU registers. This is directly accessible by the processor.

M.B. ANUSHLIN
LEENA
AP/CSE



Memory Hierarchy Design

1. Registers

Registers are small, high-speed memory units located in the CPU. They are used to store the most frequently used data and instructions. Registers have the fastest access time and the smallest storage capacity, typically ranging from 16 to 64 bits.

2. Cache Memory

Cache memory is a small, fast memory unit located close to the CPU. It stores frequently used data and instructions that have been recently accessed from the main memory. Cache memory is designed to minimize the time it takes to access data by providing the CPU with quick access to frequently used data.

3. Main Memory

Main memory, also known as RAM (Random Access Memory), is the primary memory of a computer system. It has a larger storage capacity than cache memory, but it is slower. Main memory is used to store data and instructions that are currently in use by the CPU.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Types of Main Memory

- **Static RAM:** Static RAM stores the binary information in flip flops and information remains valid until power is supplied. Static RAM has a faster access time and is used in implementing cache memory.



Dynamic RAM: It stores the binary information as a charge on the capacitor. It requires refreshing circuitry to maintain the charge on the capacitors after a few milliseconds. It contains more memory cells per unit area as compared to SRAM.

4. Secondary Storage

Secondary storage, such as hard disk drives (HDD) and solid-state drives (SSD) , is a non volatile memory unit that has a larger storage capacity than main memory. It is used to store data and instructions that are not currently in use by the CPU. Secondary storage has the slowest access time and is typically the least expensive type of memory in the memory hierarchy.

5. Magnetic Disk

Magnetic Disks are simply circular plates that are fabricated with either a metal or a plastic or a magnetized material. The Magnetic disks work at a high speed inside the computer and these are frequently used.

6. Magnetic Tape

Magnetic Tape is simply a magnetic recording device that is covered with a plastic film. Magnetic Tape is generally used for the backup of data. In the case of a magnetic tape, the access time for a computer is a little slower and therefore, it requires some amount of time for accessing the strip.

Characteristics of Memory Hierarchy

- **Capacity:** It is the global volume of information the memory can store. As we move from top to bottom in the Hierarchy, the capacity increases.
- **Access Time:** It is the time interval between the read/write request and the availability of the data. As we move from top to bottom in the Hierarchy, the access time increases.
- **Performance:** The Memory Hierarchy design ensures that frequently accessed data is stored in faster memory to improve system performance.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- **Cost Per Bit:** As we move from bottom to top in the Hierarchy, the cost per bit increases

i.e. Internal Memory is costlier than External Memory.

System-Supported Memory Standards

According to the memory Hierarchy, the system-supported memory standards are defined



below:

Level	1	2	3	4
Name	Register	Cache	Main Memory	Secondary Memory
Size	<1 KB	less than 16 MB	<16GB	>100 GB
Implementation	Multi-ports	On-chip/SRAM	DRAM(capacitor memory)	Magnetic
Access Time	0.25ns to 0.5ns	0.5 to 25ns	80 ns to 250ns	50 lakh ns
Bandwidth	20000 to 1 lakh MB	5000 to 15000	1000 to 5000	20 to 150
Manage by	Compiler	Hardware	Operating System	Operating System
Backing Mechanism	From cache	from Main Memory	from Secondary Memory	from ie

Advantages of Memory Hierarchy

- **Performance:** Frequently used data is stored in faster memory (like cache), reducing access time and improving overall system performance.
- **Cost Efficiency:** By combining small, fast memory (like registers and cache) with larger, slower memory (like RAM and HDD), the system achieves a balance between cost and performance. It saves the consumer's price and time.
- **Optimized Resource Utilization:** Combines the benefits of small, fast memory and large, cost-effective storage to maximize system performance.
- **Efficient Data Management:** Frequently accessed data is kept closer to the CPU, while less frequently used data is stored in larger, slower memory, ensuring efficient data handling.

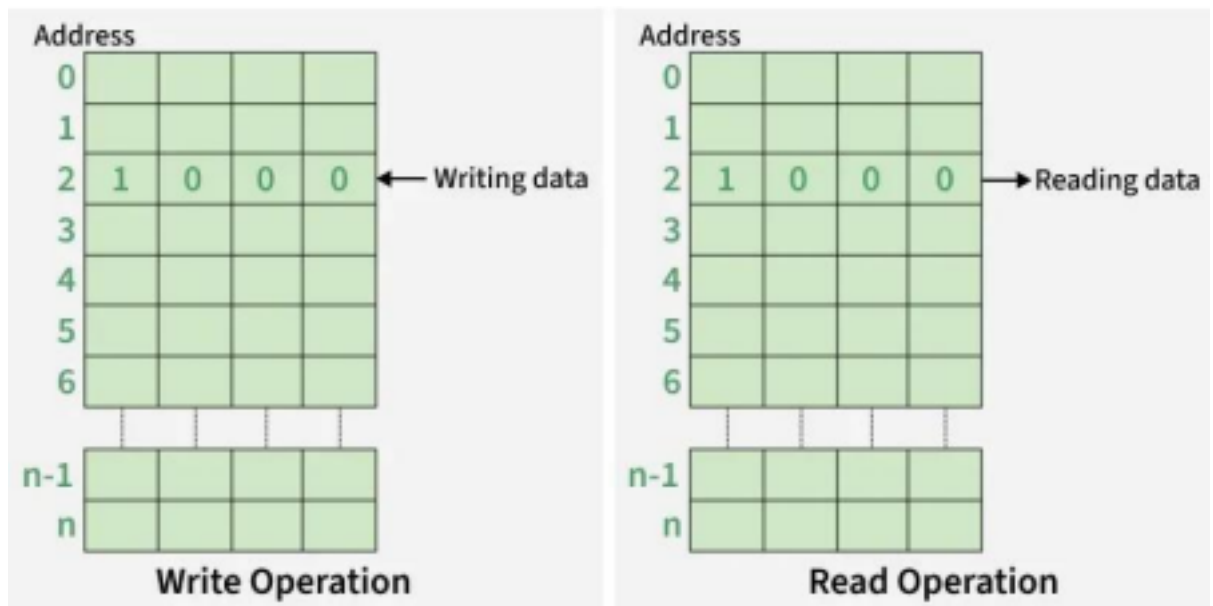
M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Disadvantages of Memory Hierarchy

- **Complex Design:** Managing and coordinating data across different levels of the hierarchy adds complexity to the system's design and operation.
- **Cost:** Faster memory components like registers and cache are expensive, limiting their size

and increasing the overall cost of the system.





- **Latency:** Accessing data stored in slower memory (like secondary or tertiary storage) increases the latency and reduces system performance.
- **Maintenance Overhead:** Managing and maintaining different types of memory adds overhead in terms of hardware and software.

Memory is an essential component of a computer system, responsible for storing data and instructions needed for processing. It enables the CPU to execute programs efficiently and ensures smooth system operation.

- **Memory Cell:** Smallest storage unit that holds 1 bit of data. Each address in memory typically refers to a byte.
- **Word & Byte:** A word is a group of bits; 1 byte = 8 bits.

- **Capacity:** Total number of bits a memory can hold.

Classification of Memory

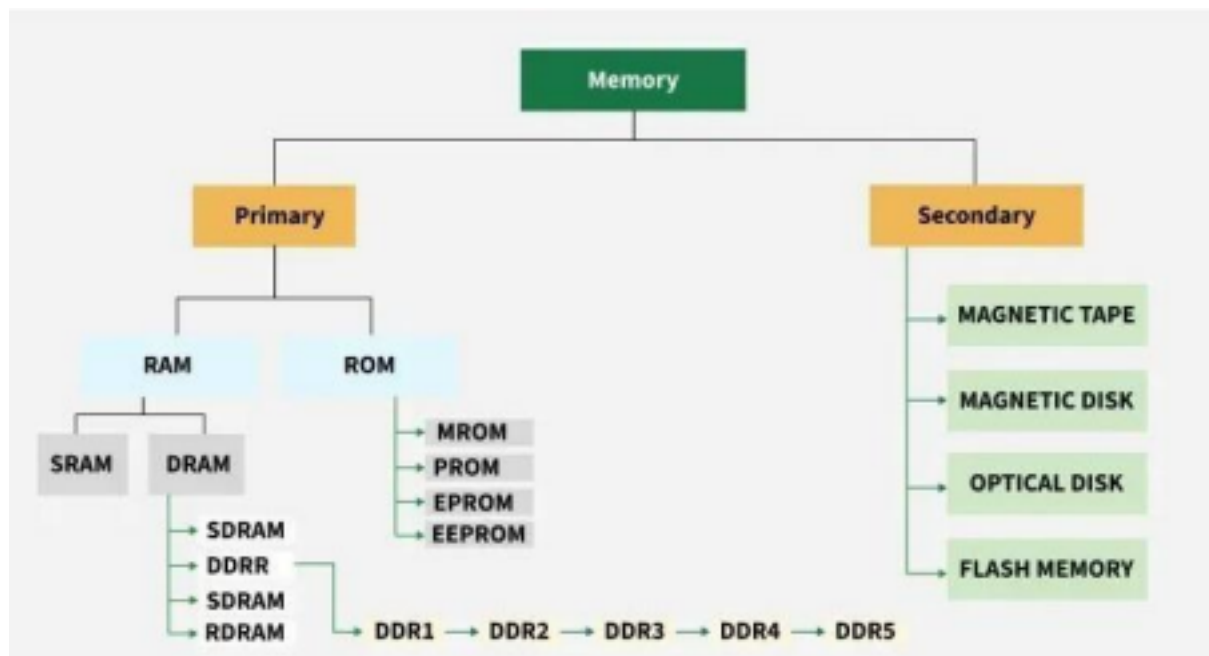
Memory is classified into primary and secondary types based on speed, accessibility, and volatility.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- **Primary Memory:** Directly accessible by the CPU; fast but limited in capacity.
- **Secondary Memory:** Used for long-term storage; slower but larger in capacity.





RAM (Random Access Memory)

RAM is the computer's main memory used for temporary storage of active programs and data. Data is lost when power is off. It provides fast CPU access, improving multitasking and performance.

Types of RAM

- **SRAM (Static RAM)**: Fast, used in cache, retains data while powered.
- **DRAM (Dynamic RAM)**: Slower, needs periodic refresh; main memory. Includes SDRAM and DDR series.

M.B. ANUSHLIN
LEENA
AP/CSE

ROM (Read-Only Memory)



ROM is non-volatile memory that stores essential instructions permanently. It holds system firmware and boot instructions.

ROM in Motherboard



It's a tiny chip but plays a HUGE role. Look for it near the battery or processor!

Types of ROM

Following are types of ROM:

- **MROM**: Pre-programmed at manufacture
- **PROM**: User-programmable once
- **EPROM**: UV-erasable
- **EEPROM**: Electrically erasable
- **Flash Memory**: Fast, used in SSDs and USB drives

Secondary Memory

Secondary memory provides long-term data storage and is not directly accessed by the CPU.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- Examples: HDD, SSD, optical discs(CD/DVD/Blu-ray), USB drives, flash cards, magnetic tapes, cloud storage.
- Characteristics: Non-volatile, slower, high capacity, used for storing operating systems, software, and user files.

Differences Among RAM, ROM, and Secondary Memory

• Following table shows major difference among three memory types:

RAM	ROM	Secondary Memory
Volatile	Non-volatile	Non-volatile
Temporary workspace	Permanent instructions	Long-term storage
Fast	Moderate	Slow
Read/Write	Mostly Read-only	Read/Write
DRAM, SRAM	PROM, EPROM, EEPROM	HDD, SSD, USB

Conversions of Units

The following table shows the standard conversions between different units of digital memory:

Name	Equal To
Bit	1 Bit
Nibble	4 Bits
Byte	8 Bits
Kilobyte	1024 Bytes

M.B. ANUSHLIN LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Name	Equal To
Megabyte	1024 Kilobytes
Gigabyte	1024 Megabytes



APPLICATIONS OF FLASH IN VARIOUS INDUSTRIES

Industrial flash modules are important across various industries, transforming technological integration and enhancing operations.

1. **Automotive:** As automotive manufacturers adopt them to improve vehicle performance and reliability, they enable quicker access to important data for advanced driver assistance systems and infotainment systems. By storing essential data for autonomous driving technologies, they ensure reliable operation and enhance safety in harsh automotive environments.
2. **Healthcare:** In the healthcare sector, industrial flash modules securely store patient records and diagnostic images, meeting stringent requirements for digital health

solutions. They facilitate fast data transfer and high reliability in medical devices like ultrasound machines and MRI scanners, supporting efficient patient care and real-time management.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

3. **Aerospace:** Important for aerospace applications, industrial flash modules securely store important flight data, software updates, and maintenance logs, offering high-speed performance and durability for harsh environmental conditions during flight. They



provide reliability and longevity essential for safe and efficient aircraft operation, supporting aviation technology advancements.

4. **Electronics:** Revolutionizing the electronics industry, industrial flash modules offer high-speed data transfer and dependable performance across various applications, from smartphones to smart home devices. They enhance processing speeds and optimize data management, ensuring consistent performance and maintaining data integrity in mission important electronic systems.
5. **Manufacturing:** In manufacturing, industrial flash modules play an important role as reliable storage solutions for essential data and applications, aiding tasks ranging from inventory management to supply chain optimization. They maintain consistent performance, even in demanding industrial settings, reducing disruptions and enabling manufacturers to uphold seamless operations. Furthermore, their adaptability allows manufacturers to modify storage capacity according to evolving business requirements, thereby improving productivity and competitiveness in the sector.

Benefits of Using Industrial Flash Modules

- Increased Storage Capacity
- Improved Performance and Reliability
- Versatility and Adaptability

CACHE MEMORY

Cache structure and mapping techniques

In modern computer systems, the speed difference between the processor and main memory (RAM) can significantly affect system performance. To bridge this gap, computers use a small, high-speed memory known as cache memory. But since cache is limited in size, the system needs a smart way to decide where to place data from main memory — and that's where cache mapping comes in.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- Cache mapping is a technique used to determine where a particular block of main memory

will be stored in the cache.

- It defines how and where that new data block from main memory will be placed inside the



Key Terminologies in Cache Mapping

Before diving into mapping techniques, let's understand some important terms:

1. **Main Memory Blocks:** The main memory is divided into equal-sized sections called blocks.
2. **Cache Lines (or Cache Blocks):** The cache memory is also divided into equal partitions called cache lines.
3. **Block Size:** The number of bytes or words stored in one block or line.
4. **Tag Bits:** A small portion of the address used to identify which block of main memory is stored in a particular cache line.
5. **Number of Cache Lines:** Determined by the ratio of Cache Size $\div$ Block Size.
6. **Number of Cache Sets:** Determined by Number of Cache Lines $\div$ Associativity (used in set-associative mapping).

Types of Cache Mapping

There are three main cache mapping techniques used in computer systems

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

1. Direct Mapping

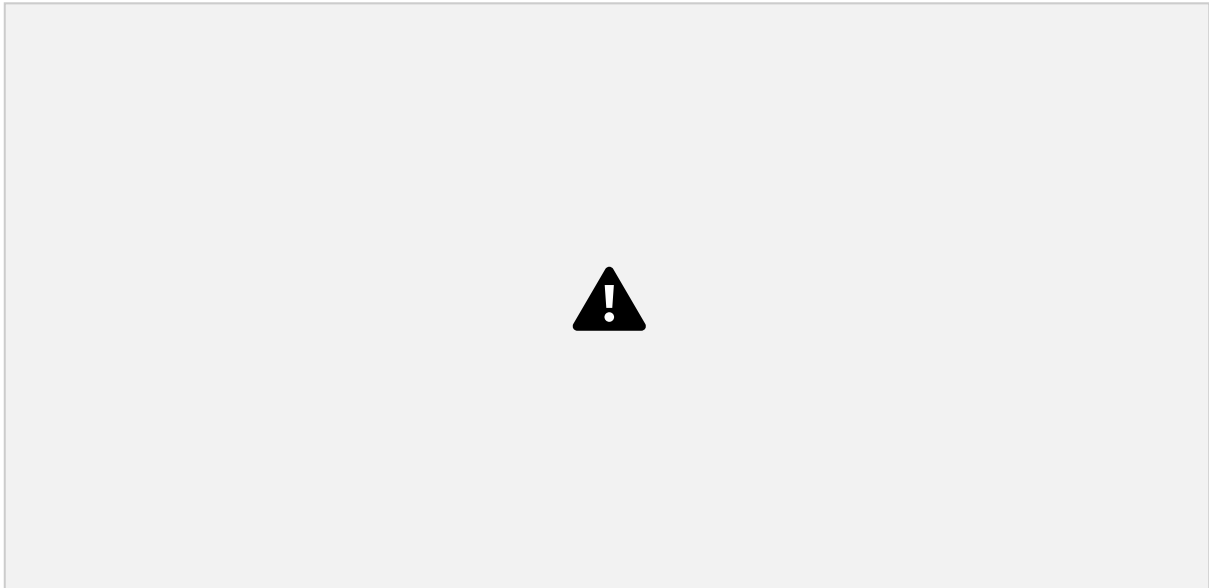
In Direct mapping, each block of main memory maps to exactly one specific cache line. The main memory address is divided into three parts:

1.
**Tag
Bits:**

Identify which block of memory is stored.

2. **Line Number:** Indicates which cache line it belongs to.

3. **Byte Offset:** Specifies the exact byte within the block.



The formula for finding the cache line is:

$$\text{Cache Line Number} = (\text{Block Number}) \text{ MOD } (\text{Number of Cache Lines})$$

Lines) 2. Fully Associative Mapping

In fully associative mapping, a memory block can be stored in any line of the cache. The address is divided into:

1. **Tag Bits:** Identify the memory block.

2. **Byte Offset:** Specifies the byte within that block.

M.B. ANUSHLIN
LEENA
AP/CSE

3.



Set-Associative Mapping

Set-associative mapping combines the benefits of both direct and fully associative mapping.

- The cache is divided into a number of sets, each containing a few lines (e.g., 2-way, 4-way set associative).
- A memory block maps to exactly one set, but can be placed in any line within that set.



2-way Set Associative Mapping

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

The address is divided into:

1. **Tag Bits:** Identify which memory block is stored.

2.
Set



Number: Determines which cache set it belongs to.

3. **Byte Offset:** Specifies the byte position.

The mapping formula is:

$$\text{Set Number} = (\text{Block Number}) \text{ MOD } (\text{Number of Sets})$$

Need for Cache Mapping

Cache mapping is essential for two main reasons:

1. **Locate Data Efficiently:** It helps the processor quickly determine whether the required data is in the cache (cache hit) or must be fetched from main memory (cache miss). 2.

Manage Data Placement: When a cache miss occurs, mapping tells the system where in the cache to place the new memory block.

Comparison Table

M.B. ANUSHLIN
LEENA
AP/CSE





Real-Life Analogy(example)

Imagine a parking lot:

- **Direct Mapping:** Each car has a fixed parking spot number.
- **Fully Associative:** Any car can park in any spot.
- **Set-Associative:** Cars are assigned to a specific section, but can park in any space within that section.

Cache Replacement policies

Cache replacement policies determine which data to remove when the cache is full, allowing the system to use its limited space more effectively. They play a key role in improving CPU speed by ensuring that frequently accessed data is available when needed.

- These policies analyze data access patterns to retain useful information and replace less relevant data.

M.B. ANUSHLIN
LEENA
AP/CSE

- By minimizing slow memory accesses, they enhance system responsiveness and overall computational efficiency.



Optimal (OPT) or Bélády's Algorithm

OPT replaces the block that will not be used for the longest time in the future. It is an ideal theoretical policy used to measure how efficient other algorithms are, but it's impossible to implement in real systems because the future memory references are unknown.

Example:

Consider a cache with 3 blocks and a reference string: 4, 0, 3, 2, 1, 3, 0,

4 1. Load 4, 0, 3 → Cache = [4, 0, 3]

2. Next 2 → Evict 4 (farthest in future) → Cache = [2, 0, 3]

3. Next 1 → Evict 2 → Cache = [1, 0, 3]

4. Next 3 → Hit → Cache = [1, 0, 3]

5. Next 0 → Hit → Cache = [1, 0, 3]

6. Next 4 → Evict 1 → Cache = [4, 0, 3]

7. Next 0 → Hit → Cache = [4, 0, 3]

□ **Result:** OPT gives the minimum possible cache misses but is impractical since future accesses are unknown.

First-In, First-Out (FIFO)

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

FIFO removes the oldest block in the cache, regardless of how frequently it's used. It's simple and easy to implement using a queue, but it may evict frequently used blocks just because they arrived early.

Example:



Cache size = 3, Reference string: 1, 2, 3, 4, 1, 2, 5, 1

1. Load 1, 2, 3 → Cache = [1, 2, 3]
2. Next 4 → Evict 1 → Cache = [4, 2, 3]
3. Next 1 → Evict 2 → Cache = [4, 1, 3]
4. Next 2 → Evict 3 → Cache = [4, 1, 2]
5. Next 5 → Evict 4 → Cache = [5, 1, 2]
6. Next 1 → Hit → Cache = [5, 1, 2]

□ **Result:** FIFO may evict frequently used items just because they were loaded earlier, leading to inefficient performance in many workloads.

Least Recently Used (LRU)

LRU replaces the block that hasn't been accessed for the longest time, based on the principle that recently used data is likely to be used again soon. It is widely used in systems where temporal locality is common.

Example:

Cache size = 3, Reference string: 1, 2, 3, 1, 4, 5, 2

1. Load 1, 2, 3 → Cache = [1, 2, 3]
2. Access 1 → Hit, recently used, no replacement → Cache = [1, 2, 3]
3. Next 4 → Replace 2 (least recently used) → Cache = [1, 4, 3]
4. Next 5 → Replace 3 → Cache = [1, 4, 5]
5. Next 2 → Replace 1 → Cache = [2, 4, 5]

□ **Result:** LRU performs well for workloads with temporal locality, such as programs reusing recently accessed variables.

Least Frequently Used (LFU)

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

LFU evicts the block that has been used the least number of times. It is effective when certain data items are accessed repeatedly, but it may keep old but once-popular data, leading to cache pollution.

Example:



Cache size = 3, Reference string: 2, 3, 2, 1, 5, 2, 4

1. Load 2 → Count 1 → Cache = [2]
2. Load 3 → Count 1 → Cache = [2, 3]
3. Access 2 → Count 2 → Cache = [2, 3]
4. Load 1 → Count 1 → Cache = [2, 3, 1]
5. Next 5 → Evict 3 (lowest count 1) → Cache = [2, 5, 1]
6. Access 2 → Count 3 → Cache = [2, 5, 1]
7. Next 4 → Evict 1 (lowest count 1) → Cache = [2, 5, 4]

□ **Result:** LFU adapts to frequently accessed data but can keep outdated blocks that were once popular, causing inefficiency.

Most Recently Used (MRU)

MRU removes the most recently accessed block. Surprisingly, this works well in scenarios where the recently used data is unlikely to be needed again soon, such as in looping over large datasets.

Example:

Cache size = 3, Reference string: 1, 2, 3, 4, 1, 2, 3, 4

1. Load 1, 2, 3 → Cache = [1, 2, 3]
2. Next 4 → Evict 3 (most recently used) → Cache = [1, 2, 4]
3. Next 1 → Hit → Cache = [1, 2, 4]
4. Next 2 → Hit → Cache = [1, 2, 4]
5. Next 3 → Evict 2 (most recently used) → Cache = [1, 3, 4]
6. Next 4 → Hit → Cache = [1, 3, 4]

□ **Result:** MRU performs better than LRU in repetitive or looping access patterns, such as scanning large arrays.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Random Replacement (RR)

RR simply selects a random block to evict when the cache is full. It's extremely easy to implement and surprisingly effective in unpredictable access patterns.

Example:



Cache size = 3, Reference string: 5, 7, 1, 2, 3, 7, 9

1. Load 5, 7, 1 $\rightarrow$ Cache = [5, 7, 1]
2. Next 2 $\rightarrow$ Evict 7 (random) $\rightarrow$ Cache = [5, 2, 1]
3. Next 3 $\rightarrow$ Evict 5 (random) $\rightarrow$ Cache = [3, 2, 1]
4. Next 7 $\rightarrow$ Evict 1 (random) $\rightarrow$ Cache = [3, 2, 7]
5. Next 9 $\rightarrow$ Evict 2 (random) $\rightarrow$ Cache = [3, 9, 7]

□ **Result:** RR is unpredictable but useful in systems like network caches where access patterns can't be easily predicted.

MULTILEVEL CACHES AND PERFORMANCE AND TUNING

Cache is a type of random access memory (RAM) used by the CPU to reduce the average time required to access data from memory. Multilevel caches are one of the techniques used to improve cache performance by reducing the miss penalty. The miss penalty refers to the additional time needed to retrieve data from the main memory when a cache miss occurs.

$$\text{Effective Access Time} = \text{Hit rate} * \text{Cache access time} + \text{Miss rate} * \text{Lower level access time}$$

Here,

- Hit Rate: The probability or percentage of finding data in the cache.
- Miss Rate: The probability or percentage of not finding data in the cache. (Miss Rate = 1 - Hit Rate)
- Cache Access Time: The time required to access data directly from the cache. •
- Lower-Level Memory Access Time: The time required to access data from lower-level memory, such as main memory or secondary storage.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Let us consider an example where the CPU makes 10 memory references to access the required information. We will analyze this scenario under the following three cases of system design:

Case 1: System Design without Cache Memory



Here the CPU directly communicates with the main memory and no caches are involved. In this case, the CPU needs to access the main memory 10 times to access the desired information.



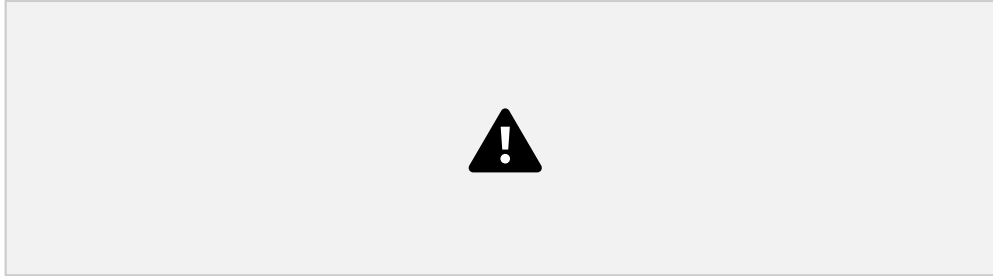
System without Cache Memory

- All memory references go directly to main memory.

- Each memory access takes **100 ns**.
- **Total time** = $10 \times 100 \text{ ns} = 1000 \text{ ns}$

Case 2: System Design with Cache Memory

Here the CPU at first checks whether the desired data is present in the Cache Memory or not i.e. whether there is a hit in cache or miss in the cache.



System with Single-Level Cache

Assume: 6 cache hits, 4 cache misses (i.e., 6 accesses are served by L1, 4 go to main memory)

• Access times:

- **L1 hits** $\rightarrow 6 \times 1 \text{ ns} = 6 \text{ ns}$
- **Misses** $\rightarrow 4 \times (1 \text{ ns} + 100 \text{ ns}) = 4 \times 101 \text{ ns} = 404 \text{ ns}$

M.B. ANUSHLIN
LEENA
AP/CSE

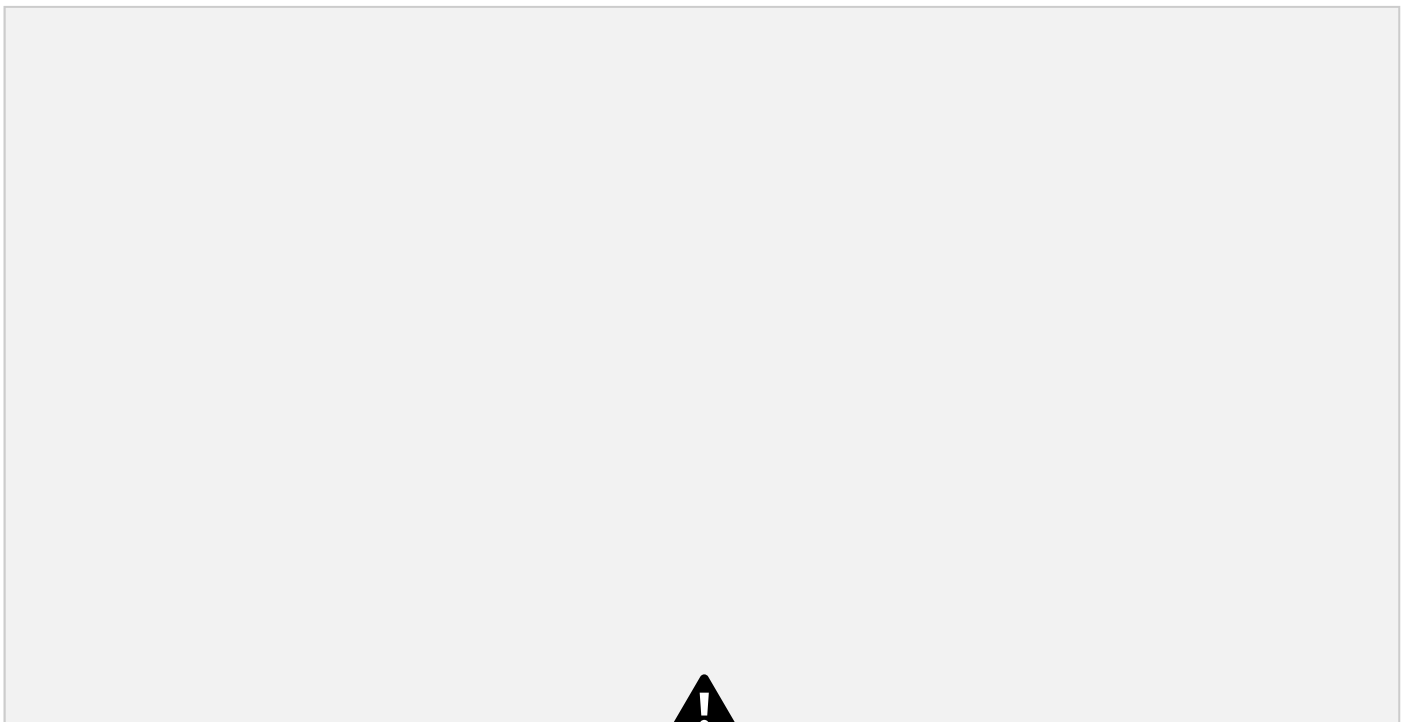
24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

(1 ns to check L1, then 100 ns for main memory fetch)

• **Total time** = $6 \text{ ns} + 404 \text{ ns} = 410 \text{ ns}$

Case 3: System Design with Multilevel Cache Memory

Here the Cache performance is optimized further by introducing multilevel Caches. As shown

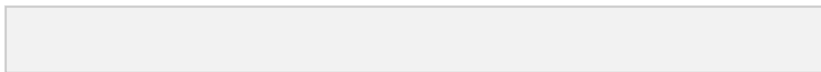


in the below figure, we are considering 2 level Cache Design. It is clear that here the Miss Penalty is reduced considerably than that in the previous case thereby improving the Performance of Cache Memory.



System Design with Multilevel Cache Memory

Average access Time for Multilevel Cache:(Tavg)



where

H1 is the Hit rate in the L1 caches. H2 is the Hit rate in the L2 cache. C1 is the Time to access information in the L1 caches. C2 is the Miss penalty to transfer information from the L2 cache to an L1 cache.

M is the Miss penalty to transfer information from the main memory to the L2 cache. Assume: 6 L1 hits , 3 L1 misses that hit in L2 ,1 full miss (goes to main memory)

• Access times:

- **L1 hits** $\rightarrow 6 \times 1 \text{ ns} = \mathbf{6 \text{ ns}}$
- **L2 hits** $\rightarrow 3 \times (1 \text{ ns} + 5 \text{ ns}) = 3 \times 6 \text{ ns} = \mathbf{18 \text{ ns}}$
- **Main memory** $\rightarrow 1 \times (1 \text{ ns} + 5 \text{ ns} + 100 \text{ ns}) = \mathbf{106 \text{ ns}}$

• **Total time** = $6 \text{ ns} + 18 \text{ ns} + 106 \text{ ns} = \mathbf{130 \text{ ns}}$

NOTE : We can observe from the above 3 cases that we are trying to decrease the number of Main

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Memory References and thus decreasing the Miss Penalty in order to improve the overall System Performance. Also, it is important to note that in the Multilevel Cache Design, L1 Cache is attached to the CPU and it is small in size but fast. Although, L2 Cache is attached to



the Primary Cache i.e. L1 Cache and it is larger in size and slower but still faster than the Main Memory.

EXAMPLE

Find the Average memory access time for a processor with a 2 ns clock cycle time, a miss rate of 0.04 misses per instruction, a missed penalty of 25 clock cycles, and a cache access time (including hit detection) of 1 clock cycle. Also, assume that the read and write miss penalties are the same and ignore other write stalls.

Solution:



Advantages of Multilevel Cache Organization

- **Reduced access time:** By having multiple levels of cache, the access time to frequently accessed data is greatly reduced. This is because the data is first searched for in the smallest, fastest cache level and if not found, it is searched for in the next larger, slower cache level.
- **Improved system performance:** With faster access times, system performance is improved as the CPU spends less time waiting for data to be fetched from memory.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- **Lower cost:** By having multiple levels of cache, the total amount of cache required can be minimized. This is because the faster, smaller cache levels are more expensive per unit of memory than the larger, slower cache levels.
- **Energy efficiency:** Since the data is first searched for in the smallest cache level, it is more

likely that the data will be found there, and this reduces the power consumption of the system.

Disadvantages of Multilevel Cache Organization

- **Complexity:** The addition of multiple levels of cache increases the complexity of the cache hierarchy and the overall system. This complexity can make it more difficult to design, debug and maintain the system.
- **Higher latency for cache misses:** If the data is not found in any of the cache levels, it will have to be fetched from the main memory, which has a higher latency. This delay can be especially noticeable in systems with deep cache hierarchies.
- **Higher cost:** While having multiple levels of cache can reduce the overall amount of cache required, it can also increase the cost of the system due to the additional cache hardware required.
- **Cache coherence issues:** As the number of cache levels increases, it becomes more difficult to maintain cache coherence between the various levels. This can result in inconsistent data being read from the cache, which can cause problems with the overall system operation.

VIRTUAL MEMORY: PAGING

Virtual memory is a memory management technique used by operating systems to give the appearance of a large, continuous block of memory to applications, even if the physical memory (RAM) is limited and not necessarily allocated in contiguous manner.

- A program doesn't need to be fully loaded in memory to run. Only the needed parts are loaded.
- Programs can be bigger than the physical memory available in the system. • Virtual memory creates the illusion of a large memory, even if the actual memory (RAM) is small.
- It uses both RAM and disk storage to manage memory, loading only parts of programs into RAM as needed.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- This allows the system to run more programs at once and manage memory more efficiently.



How Virtual Memory Works

• Virtual memory uses both hardware and software to manage memory. • When a program runs, it uses virtual addresses (not real memory locations). • The computer system converts these virtual addresses into physical addresses (actual locations in RAM) while the program runs.

Types of Virtual Memory

Virtual memory is managed by the Memory Management Unit (MMU), which is often built into the CPU. The CPU generates virtual addresses that the MMU translates into physical addresses. There are two main types of virtual memory:

1. Paging
2. Segmentation

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

1. Paging

Paging divides memory into small fixed-size blocks called pages. When the computer runs out



of RAM, pages that aren't currently in use are moved to the hard drive, into an area called a swap file. Here,

- The swap file acts as an extension of RAM.
- When a page is needed again, it is swapped back into RAM, a process known as page swapping.
- This ensures that the operating system (OS) and applications have enough memory to run.

Page Fault Service Time: The time taken to service the page fault is called page fault service time. The page fault service time includes the time taken to perform all the above steps. • Let

Main memory access time is: mm

- Page fault service time is: ss

- Page fault rate is : pp

- Then, Effective memory access time = $(p*s)+(1-p)*m$ **Page and Frame:**

Page is a fixed size block of data in virtual memory and a frame is a fixed size block of physical memory in RAM where these pages are loaded.

- Think of a page as a piece of a puzzle (virtual memory) While, a frame as the spot where it fits on the board (physical memory).

- When a program runs its pages are mapped to available frames so the program can run even if the program size is larger than physical memory.

2. Segmentation

Segmentation divides virtual memory into segments of different sizes. Segments that aren't currently needed can be moved to the hard drive. Here,

- The system uses a segment table to keep track of each segment's status, including whether it's in memory, if it's been modified and its physical address.

- Segments are mapped into a process's address space only when needed.

Applications of Virtual memory

- **Increased Effective Memory:** It enables a computer to have more memory than the physical memory using the disk space. This allows for the running of larger applications.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- **Memory Isolation:** Virtual memory allocates a unique address space to each process, such separation increases safety and reliability based on the fact that one process cannot interact with another.

- **Efficient Memory Management:** Virtual memory also helps in better utilization of the

physical memories through methods that include paging and segmentation. • **Simplified**

Program Development: For ease of programmers, they can program 'as if' there is one big block of memory and this makes the programming easier and more efficient in delivering more complex applications.

Management of Virtual Memory

Here are 5 key points on how to manage virtual memory:

1. Adjust the Page File Size

- **Automatic Management:** All contemporary OS including Windows contain the auto configuration option for the size of the empirical page file. But depending on the size of the RAM, they are set automatically, although the user can manually adjust the page file size if required.
- **Manual Configuration:** For tuned up users, the setting of the custom size can sometimes boost up the performance of the system. The initial size is usually advised to be set to the minimum value of 1.

2. Place the Page File on a Fast Drive

- **SSD Placement:** If this is feasible, the page file should be stored in the SSD instead of the HDD as a storage device. It has better read and write times and the virtual memory may prove beneficial in an SSD.
- **Separate Drive:** Regarding systems having multiple drives involved, the page file needs to be placed on a different drive than the OS and that shall in turn improve its performance.

3. Monitor and Optimize Usage

- **Performance Monitoring:** Employ the software tools used in monitoring the performance of the system in tracking the amounts of virtual memory.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- **Regular Maintenance:** Make sure there is no toolbar or other application running in the background, take time and uninstall all the tool bars to free virtual memory.
4. Keep Virtual

Memory Enabled

- Even with large RAM (e.g., 16GB+), virtual memory should generally remain enabled, as



the OS relies on it for efficient memory management and system stability.

5. Optimize System Settings

- **System Configuration:** Change some general properties of the system concerning virtual memory efficiency. This also involves enabling additional control options in Windows.
- **Regular Updates:** Ensure that your drivers are run in their newest version because new releases contain some enhancements and issues regarding memory management.

Benefits of Using Virtual Memory

- **Supports Multiprogramming & Larger Programs :** Virtual memory allows multiple processes to reside in memory at once by using demand paging. Even programs larger than physical memory can be executed efficiently.
- **Maximizes Application Capacity :** With virtual memory, systems can run more applications simultaneously, including multiple large ones. It also allows only portions of programs to be loaded at a time, improving speed and reducing memory overhead.
- **Eliminates Physical Memory Limitations :** There's no immediate need to upgrade RAM as virtual memory compensates using disk space.
- **Boosts Security & Isolation :** By isolating the memory space for each process, virtual memory enhances system security. This prevents interference between applications and reduces the risk of data corruption or unauthorized access.
- **Improves CPU & System Performance:** Virtual memory helps the CPU by managing logical partitions and memory usage more effectively. It allows for cost-effective, flexible resource allocation, keeping CPU workloads optimized and ensuring smoother multitasking.
- **Enhances Memory Management Efficiency :** Virtual memory automates memory allocation, including moving data between RAM and disk without user intervention. It

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

also avoids external fragmentation, using more of the available memory effectively and simplifying OS-level memory management.





TRANSLATION LOOKASIDE BUFFER (TLB) IN PAGING

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Operating systems use paging to map virtual addresses to physical addresses efficiently. Each process maintains its own Page Table, which contains Page Table Entries (PTEs) responsible



for translating virtual page numbers into physical frame numbers. This mechanism enables flexible memory allocation and prevents processes from interfering with each other's memory space.

- Paging divides memory into fixed-size pages and frames.
- Each process has a separate Page Table for address translation.
- Page Table Entries (PTEs) store the frame number and status bits.
- Paging improves memory protection, isolation, and efficient utilization



The Problem of Access Time in Paging

When the CPU generates a virtual address, it must be translated into a physical address to access data in main memory. The naive approach stores the entire page table in main memory. Therefore, for every memory access, the following two memory accesses are required:

1. Access the page table in main memory to get the frame number.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

2. Access the actual data in the main memory frame.

Note: This leads to a performance problem, as every memory access now requires two memory accesses, causing a significant slowdown.

Why Not Store Page Table in Registers?



Initially, it seemed ideal to store the page table in the high-speed CPU registers to speed up lookups since registers provide the fastest access time. However:

- Registers are very limited in size and can store only a small number of entries.
- For large processes, the page table can be very large. For example, in a 32-bit system with 4 KB page size, the page table may contain around 1 million entries.

Therefore, this approach is impractical and the entire page table is kept in main memory instead.

- To find the frame number: The CPU first accesses the page table in main memory to obtain the frame number similar to the page number.

- To go to the address specified by the frame number: After getting the frame number, the CPU accesses the actual memory location (frame) to read/write the data. **How Does the TLB Work?**

When the CPU generates a virtual address, it first looks up the page number in the TLB.

- **TLB Hit:** It is the situation when, the page number is found in the TLB, the corresponding frame number is retrieved instantly.
- **TLB Miss:** It is the situation when, the page number is not found in the TLB, the CPU accesses the page table in main memory.

Step-by-Step Process

Steps in TLB Hit:

1. CPU generates a virtual (logical) address.
2. It is checked in TLB (present).
3. The corresponding frame number is retrieved, which now tells where the main memory page lies.

Steps in TLB miss:

1. CPU generates a virtual (logical) address.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

2. It is checked in TLB (not present).
3. Now the page number is matched to the page table residing in the main memory (assuming the page table contains all PTE).
4. The corresponding frame number is retrieved, which now tells where the main memory

page lies.

5. The TLB is updated with new PTE (if space is not there, one of the replacement techniques comes into the picture i.e either FIFO, LRU or MFU etc).

Effective memory access time(EMAT)

TLB is used to reduce adequate memory access time as it is a high-speed associative cache.

$$EMAT = h(c+m) + (1-h)(c+2m)$$

where:

- h = Hit ratio of the TLB
- m = Main memory access time
- c = TLB access time
-

Note: A higher hit ratio h significantly reduces EMAT, making the system efficient.

Benefits of Using TLB in Paging

- **Faster Address Translation:** Reduces the need for repeated page table accesses.
- **Better Performance:** Reduces average memory access time dramatically.
- **Efficient Memory Utilization:** Only the most frequently used PTEs are cached in the small, fast TLB.
- **Scalability:** Especially useful for large address spaces in 64-bit architectures.

Note: Without the TLB, every memory access would require multiple memory lookups, severely degrading performance.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

MEMORY MANAGEMENT IN OPERATING SYSTEM

Memory management is the process of controlling and organising a computer's memory by allocating portions, called blocks, to different executing programmes to improve the overall

system performance.



- The most important function of an operating system is to manage primary memory. •
- Supports multiple processes simultaneously in memory.
- Protects processes from unauthorised access.
- Enables swapping and virtual memory efficiently.



Techniques in Memory Allocation

Used by an operating system to efficiently allocate, utilize, and manage memory resources for processes. Various techniques help the operating system manage memory effectively. They can be broadly categorized into:

M.B. ANUSHLIN
LEENA
AP/CSE



Swapping

Swapping is a memory management technique where processes are temporarily moved between main memory and secondary storage to free up memory for other processes. •

Allows multiple processes to run efficiently.

• Lower-priority processes can be swapped out for higher-priority ones. •

Swapped-out processes resume when loaded back into memory.

- Transfer time depends on the amount of data moved.

□ **Contiguous Memory Allocation**

Each process is allocated a single continuous block of memory. All instructions and data of a process are stored in adjacent memory locations.

Single Contiguous Memory Allocation

Simplest form of memory management. In this technique, the main memory is divided into two parts:

- One part is reserved for the Operating System
- The remaining part is allocated to a single user process

Characteristics

- Only one user process can reside in memory at a time
- The operating system occupies a fixed portion of memory
- No multiprogramming is possible

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- Simple to implement and manage

Advantages

- Simple memory management
- No fragmentation issues



Disadvantages

- Poor memory utilization
- No support for multitasking or multiprogramming

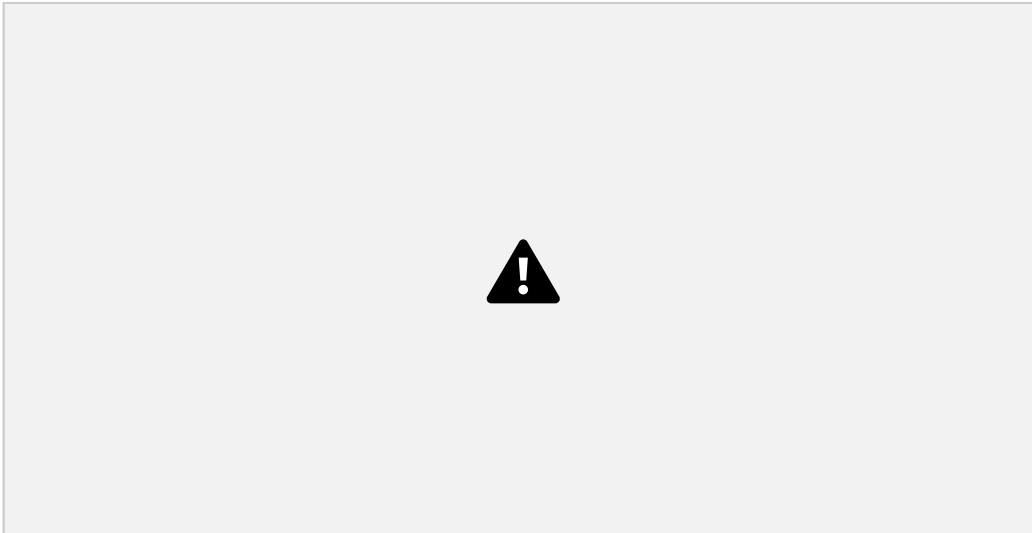
Partitioned Memory Allocation

Main memory is divided into multiple contiguous partitions, and each partition can hold one process. This technique supports multiprogramming.

Partitioned memory allocation is further classified into:

Fixed Partition Allocation

- Memory is divided into a fixed number of partitions
- Each partition has a fixed size
- Each partition can store only one process
- Leads to internal fragmentation
- Once partitions are defined operating system keeps track of the status of memory partitions
it is done through a data structure called a partition table.



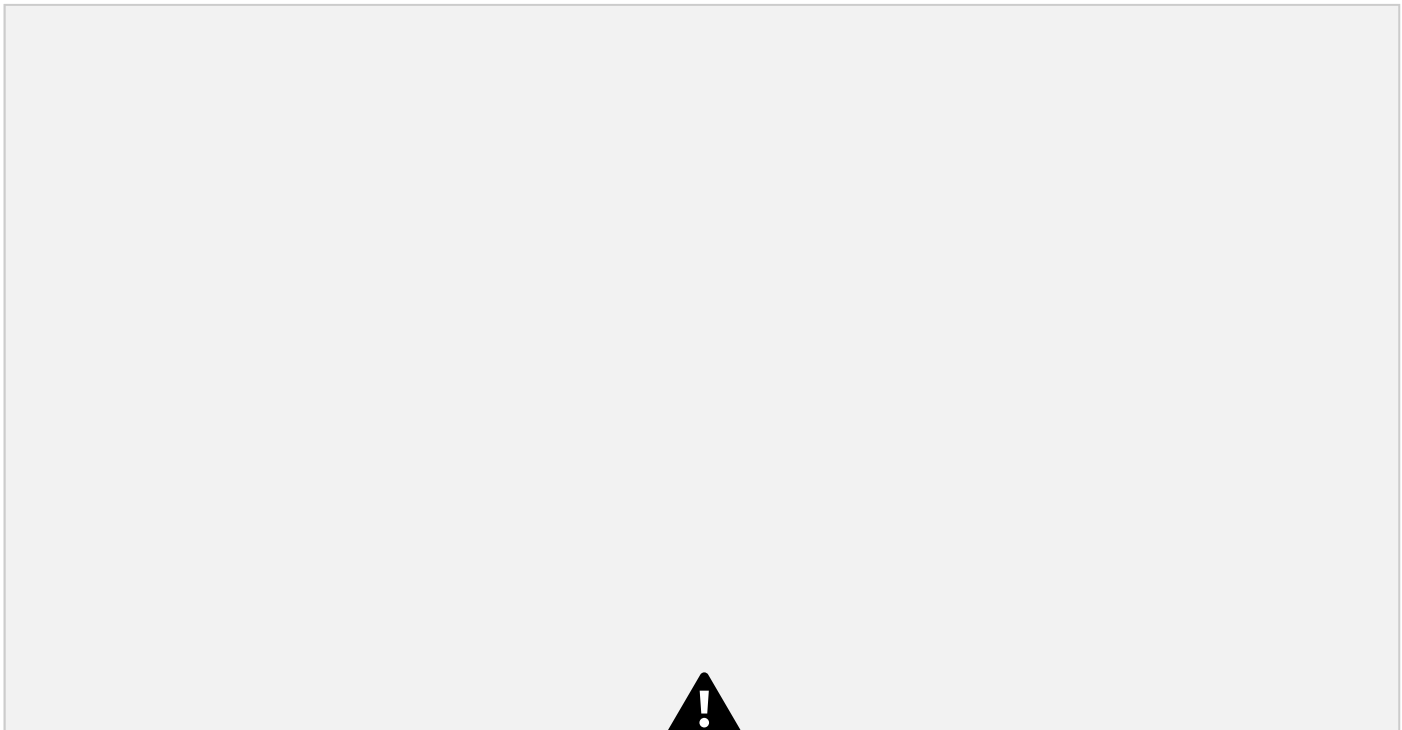
Variable Partition Allocation

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

- Memory is divided into partitions dynamically based on process size •
- Reduces internal fragmentation
- Suffers from external fragmentation

Advantages



Supports multiprogramming

- Better memory utilization compared to single contiguous allocation

Disadvantages

- Fragmentation issues
- Complex memory management compared to single contiguous allocation □

Non-Contiguous Memory Allocation

Memory management technique in which a process is divided into smaller parts and these parts are stored in different, non-adjacent locations in main memory. Unlike contiguous allocation, the entire process does not need to be placed in a single continuous block of memory.

This technique is widely used in modern operating systems because it improves memory utilization and reduces fragmentation problems.

Features of Non-Contiguous Memory Allocation

- A process can be stored in multiple memory locations
- Improves utilization of available memory
- Reduces external fragmentation
- Requires address translation using hardware support (MMU)

Advantages

- Better memory utilization
- Supports large programs
- Eliminates the need for contiguous free memory

Disadvantages

- More complex than contiguous allocation
- Additional overhead for address translation
- Requires extra memory for tables (page table / segment table)

Techniques Used in Non-Contiguous Memory Allocation

1. **Paging:** Divides a process into fixed-size pages and memory into frames of the same size
2. **Segmentation:** Divides a process into logical segments of variable size such as code, data, and stack

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

3. **Segmentation with Paging:** Combines logical segmentation with paging to reduce

fragmentation

Memory Management Mechanisms



Virtual Memory

- Let a program run even if it is larger than physical RAM.
- Uses disk as an extension of main memory.

Page Replacement Algorithms(PRA)

- **PRA:** Decide which page to remove from memory when it is full.
- **FIFO:** Remove the page that came first.

- **LRU:** Remove the page least recently used.
- **Optimal:** Remove the page not needed for the longest future time.
- **LFU:** Remove the page used least frequently.

Demand Paging

- Demand Paging loads only the pages a process actually needs into memory. •
- Reduces unnecessary memory usage and I/O.

Memory Management Problems

Fragmentation

- Fragmentation Occurs when processes are loaded into and removed from memory, resulting in unused memory spaces that cannot be efficiently utilized.
- **Internal Fragmentation:** Occurs when a process is allocated more memory than it actually needs, leading to wasted space inside the allocated memory block.
- **External Fragmentation:** Occurs when free memory is divided into small, scattered blocks, making it impossible to allocate a large contiguous block even though enough total free memory exists.

M.B. ANUSHLIN
LEENA
AP/CSE

24CS303-24CS303-COMPUTER ARCHITECTURE & ORGANIZATION

Thrashing

- Thrashing occurs when the system spends most of its time swapping pages between memory

and disk instead of executing processes, causing very low CPU utilization. **Memory**

Allocation Strategies

Fixed Partition Allocation: Memory is divided into fixed-sized partitions, and each partition can hold only one process. The OS keeps track of free and occupied partitions using a partition table.

Dynamic Partition Allocation: Memory is divided into variable-sized partitions based on the size of the processes. This helps avoid wastage of memory but can result in fragmentation.

Placement Algorithms: When allocating memory, the OS uses placement algorithms to decide which free block should be assigned to a process:

- **First Fit:** Allocates the first available partition large enough to hold the process.
- **Best Fit:**

Allocates the smallest available partition that fits the process, reducing wasted space.

- **Worst Fit:** Allocates the largest available partition, leaving the largest remaining space.
-

Next Fit: Similar to First Fit but starts searching for free memory from the point of the last allocation.

M.B. ANUSHLIN
LEENA
AP/CSE